

AFRILANG Stdlib

std/c/graph topo

Bibliothèque AFRILANG `std/c/graph topo` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : topoSort, hasCy

```
`import "std/c/graph topo" . `use graph topo`
```

- `topoSort(adj list number, n number) ? list number`
- `hasCycleDirected(adj list number, n number) ? bool`
- `inDegrees(adj list number, n number) ? list number`
- `outDegrees(adj list number, n number) ? list number`
- `sources(adj list number, n number) ? list number`
- `sinks(adj list number, n number) ? list number`
- `isDag(adj list number, n number) ? bool`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphtopo` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : topoSort, hasCy

```
`import "std/c/graphtopo" . `use graphtopo`  
- `topoSort(adj list number, n number) ? list number`  
- `hasCycleDirected(adj list number, n number) ? bool`  
- `inDegrees(adj list number, n number) ? list number`  
- `outDegrees(adj list number, n number) ? list number`  
- `sources(adj list number, n number) ? list number`  
- `sinks(adj list number, n number) ? list number`  
- `isDag(adj list number, n number) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphtopo"  
use graphtopo
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? topoSort(adj list number, n number) ? list  
? hasCycleDirected(adj list number, n number) ? bool  
? inDegrees(adj list number, n number) ? list  
? outDegrees(adj list number, n number) ? list  
? sources(adj list number, n number) ? list  
? sinks(adj list number, n number) ? list  
? isDag(adj list number, n number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/graphtopo"  
use graphtopo  
  
create result = topoSort(/* args */)   
say result  
  
create result = hasCycleDirected(/* args */)   
say result  
  
create result = inDegrees(/* args */)   
say result  
  
create result = outDegrees(/* args */)   
say result  
  
create result = sources(/* args */)   
say result  
  
create result = sinks(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/graphtopo"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

module graphtopo

```
function topoSort(adj list number, n number) returns list number
end

function hasCycleDirected(adj list number, n number) returns bool
end

function inDegrees(adj list number, n number) returns list number
end

function outDegrees(adj list number, n number) returns list number
end

function sources(adj list number, n number) returns list number
end

function sinks(adj list number, n number) returns list number
end

function isDag(adj list number, n number) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphtopo` (tier complex, category complex). Exports 7 function(s): topoSort, hasCycleDirected,

```
`import "std/c/graphtopo" . `use graphtopo`  
- `topoSort(adj list number, n number) ? list number`  
- `hasCycleDirected(adj list number, n number) ? bool`  
- `inDegrees(adj list number, n number) ? list number`  
- `outDegrees(adj list number, n number) ? list number`  
- `sources(adj list number, n number) ? list number`  
- `sinks(adj list number, n number) ? list number`  
- `isDag(adj list number, n number) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphtopo"  
use graphtopo
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? topoSort(adj list number, n number) ? list  
? hasCycleDirected(adj list number, n number) ? bool  
? inDegrees(adj list number, n number) ? list  
? outDegrees(adj list number, n number) ? list  
? sources(adj list number, n number) ? list  
? sinks(adj list number, n number) ? list  
? isDag(adj list number, n number) ? bool
```

4. Usage examples

```
import "std/c/graphtopo"  
use graphtopo  
  
create result = topoSort(/* args */)   
say result  
  
create result = hasCycleDirected(/* args */)   
say result  
  
create result = inDegrees(/* args */)   
say result  
  
create result = outDegrees(/* args */)   
say result  
  
create result = sources(/* args */)   
say result  
  
create result = sinks(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphtopo"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphtopo
```

```
function topoSort(adj list number, n number) returns list number
end

function hasCycleDirected(adj list number, n number) returns bool
end

function inDegrees(adj list number, n number) returns list number
end

function outDegrees(adj list number, n number) returns list number
end

function sources(adj list number, n number) returns list number
end

function sinks(adj list number, n number) returns list number
end

function isDag(adj list number, n number) returns bool
end
end
```